

Prompts Are Programs Too! Understanding How Developers Build Software Containing Prompts

Nikhil Anand

April 13, 2026

The rapid adoption of generative AI and foundation models has fundamentally reshaped how software is developed, introducing natural language prompts as a core mechanism for building intelligent features. Rather than serving only as auxiliary tools, these prompts increasingly function as executable components embedded within applications, enabling tasks such as code generation, debugging, and information retrieval. This shift has led to the emergence of “prompt programming,” where developers design prompts that accept variable inputs and produce outputs at runtime, similar to traditional programs but expressed in natural language. Despite the widespread deployment of such systems across major platforms and tools, there remains limited understanding of how developers actually construct, refine, and integrate these prompt-based components into software systems.

To address this gap, the paper investigates prompt programming as a distinct programming paradigm, emphasizing its unique challenges and workflows compared to traditional software development. Using a qualitative approach based on Straussian grounded theory, the study explores how developers iteratively build mental models of foundation model behavior through experimentation and interaction. Unlike conventional programming, where logic is deterministic and transparent, prompt programming involves dealing with ambiguity, stochastic outputs, and evolving model capabilities. This results in a development process that is highly iterative, exploratory, and dependent on empirical observation. By analyzing insights from interviews with developers across diverse contexts, the work provides a structured understanding of prompt programming practices and highlights the need for new tools, methodologies, and educational approaches tailored to this emerging paradigm.

1 Motivation

The motivation for this paper stems from the rapid adoption of foundation models and generative AI systems in software development, which are fundamentally reshaping how developers build and interact with software. Systems such as ChatGPT and GPT-4 have enabled developers to integrate powerful capabilities through natural language interfaces [1, 2], while models like DALL·E extend this paradigm to multimodal generation [3]. Additionally, large language models trained on code have demonstrated the ability to generate and assist with programming tasks [4], further accelerating their integration into development workflows. This shift highlights that AI is no longer just a supporting tool but a core component of modern software systems, raising important questions about how developers design and interact with these systems.

A key motivation of the paper is the observation that natural language prompts are increasingly functioning as executable artifacts, effectively introducing a new programming paradigm. Unlike traditional programming, which is based on deterministic logic and explicit instructions, prompt-based systems operate in a probabilistic and opaque environment, where outputs depend on model behavior and training. While prior work has explored the usability and effectiveness of AI programming assistants [5], these studies primarily focus on productivity and user experience. They do not address how developers conceptualize prompts as part of the system itself. This creates a gap in understanding, motivating the need to treat prompt programming as a first-class programming activity rather than an auxiliary interaction mechanism.

Another important motivation is the lack of transparency and predictability in foundation models, which introduces new challenges for developers. Techniques such as reinforcement learning from human feedback have improved model alignment and usability [6], but they do not eliminate uncertainty in outputs. As a result, developers must iteratively experiment with prompts, interpret outputs, and refine their inputs without guarantees of correctness, leading to a workflow that is fundamentally different from traditional software engineering. This motivates the need to study how developers build mental models of these systems and adapt their problem-solving strategies in response to uncertainty.

Finally, the paper is motivated by the broader goal of understanding how AI systems reshape the nature of programming itself. As developers increasingly rely on prompts to specify behavior, the boundary between programming and interaction becomes blurred. This shift has implications not only for tooling and workflows but also for how developers reason about systems, debug issues, and ensure reliability. By investigating prompt programming practices, the paper aims to provide insights that can guide the development of new abstractions, methodologies, and tools tailored to AI-driven software development, addressing the limitations of existing paradigms.

2 Methodology

2.1 Grounded Theory

The study adopts a Straussian grounded theory approach to systematically investigate prompt programming practices, enabling the authors to derive insights directly from empirical data rather than imposing predefined hypotheses. Grounded theory is particularly suited for exploring emerging phenomena where existing theoretical frameworks are insufficient, making it appropriate for studying prompt programming as a novel paradigm. The authors follow established grounded theory procedures [7], beginning with open coding to identify initial concepts, followed by axial coding to relate categories, and finally selective coding to synthesize a coherent theoretical understanding. This structured process allows the researchers to iteratively refine concepts and build a theory grounded in developer experiences.

During open coding, interview transcripts are analyzed line-by-line to extract meaningful units, capturing how developers describe their interactions with prompts and foundation models. These codes are then grouped and refined through axial coding, where relationships between categories—such as prompt construction, iteration, and evaluation—are identified. The process is highly iterative, involving constant comparison across data points to ensure consistency and depth in emerging themes [7]. This enables the authors to uncover patterns in how developers approach prompt programming, particularly in terms of experimentation and adaptation.

In the final stage of selective coding, the authors integrate the identified categories into a cohesive framework that explains prompt programming as a development practice. This phase focuses on identifying a central phenomenon and connecting all categories around it, resulting in a theoretical model that captures the iterative, exploratory, and feedback-driven nature of prompt programming. The use of grounded theory ensures that the findings are not only descriptive but also explanatory, providing a foundation for future research and tool development in this space.

2.2 Participants

The study involves a diverse set of participants with experience using foundation models in real-world development contexts, ensuring that the findings reflect practical and varied perspectives. Participants were selected to capture a range of roles, expertise levels, and application domains, including professional developers and practitioners who actively integrate prompts into their workflows. This diversity allows the study to capture different usage patterns and challenges associated with prompt programming, rather than being limited to a narrow subset of users.

Additionally, the selection criteria emphasize participants who have hands-on experience with prompt-based systems, ensuring that the insights are grounded in actual practice rather than theoretical understanding. This enables the authors to analyze how prompt programming manifests across different contexts, including varying levels of expertise and familiarity with AI tools.

2.3 Interview Protocol

The authors employ a semi-structured interview protocol to gather in-depth qualitative data on developer experiences with prompt programming. This approach allows for consistency across interviews while still providing flexibility to explore participant-specific insights. The interviews focus on understanding how developers construct prompts, iterate on them, and evaluate outputs, enabling the researchers to capture the end-to-end workflow of prompt programming in practice.

The protocol is designed to encourage participants to reflect on their experiences, challenges, and strategies when working with foundation models. By combining structured questions with open-ended discussions, the study is able to elicit rich, detailed responses that reveal both common patterns and individual variations in prompt programming practices. This approach supports the grounded theory methodology by providing data that is both systematic and exploratory.

3 Key Findings

3.1 Properties of Prompt Programming

3.1.1 Programmer

Prompt programming places the programmer in a fundamentally different role compared to traditional software development. Instead of writing deterministic logic, the programmer must interact with a foundation model whose behavior is not fully transparent or predictable. This requires developers to engage in an exploratory workflow where they iteratively refine prompts based on observed outputs. As a result, the programmer’s role shifts toward **experimenting, interpreting outputs, and adapting strategies based on feedback**, rather than strictly implementing predefined logic. This highlights a transition from precise control to guided interaction with the model.

Another defining property is the need for programmers to construct and refine mental models of how the foundation model behaves. Since the internal workings of the model are largely opaque, developers rely on repeated interactions to infer how different prompt formulations influence outputs. This process introduces a new cognitive dimension, where programmers must **reason about probabilistic behavior and implicit patterns learned by the model**, rather than explicit code execution paths. Consequently, prompt programming involves a blend of intuition, experimentation, and experience-driven learning.

Additionally, programmers must manage uncertainty and variability in outputs, which contrasts sharply with the deterministic nature of traditional programming. Even well-crafted prompts may produce inconsistent results across executions, requiring developers to design prompts that are robust to such variation. This leads to practices focused on **iterative refinement, validation through repeated trials, and heuristic-based adjustments**, emphasizing adaptability over precision. The programmer thus becomes an active participant in a feedback loop with the model, continuously shaping and reshaping prompts.

3.1.2 Foundation Model

The foundation model serves as a central computational entity whose behavior is both powerful and unpredictable. Unlike traditional systems with well-defined logic, foundation models generate outputs based on learned patterns from large-scale training data, leading to responses that are context-sensitive but not guaranteed to be consistent. This makes the model inherently probabilistic, requiring developers to account for variability and ambiguity in its outputs. As a result, the model is best understood as a system that supports **flexible, context-driven generation but lacks deterministic guarantees**, fundamentally influencing how prompt programming is approached.

3.1.3 Prompt

Prompts act as the primary interface through which programmers communicate intent to the foundation model, effectively serving as executable specifications. Unlike traditional code, prompts are written in natural language and must encode both the task and the context required for the model to generate

appropriate outputs. This makes prompts inherently expressive but also ambiguous, requiring careful construction. Developers must therefore focus on **crafting precise yet flexible instructions that guide the model’s behavior**, balancing clarity with adaptability.

Another important property of prompts is their iterative nature. Developers rarely arrive at an optimal prompt on the first attempt; instead, they refine prompts through cycles of testing and observation. Small changes in wording, structure, or examples can significantly impact the output, making prompt design a highly sensitive process. This leads to workflows centered around **incremental refinement, experimentation, and empirical evaluation of outputs**, rather than static implementation.

Finally, prompts often incorporate examples, constraints, or structured formats to improve output quality, reflecting a shift toward designing inputs that shape model reasoning. This includes techniques such as few-shot prompting and explicit instruction formatting, which help guide the model toward desired behaviors. As a result, prompts become more than simple queries—they evolve into structured artifacts that encode logic and expectations. This highlights the role of prompts as **dynamic, evolving components that encapsulate both intent and interaction strategy** in prompt programming.

3.2 Prompt Programming Process

3.2.1 Design

The design phase in prompt programming involves formulating an initial prompt that captures the developer’s intent while accounting for the behavior of the foundation model. Unlike traditional design processes that rely on formal specifications, prompt design is inherently exploratory, requiring developers to anticipate how the model might interpret natural language instructions. This leads to a process where developers must **translate abstract goals into structured natural language inputs**, often incorporating task descriptions, constraints, and contextual information to guide the model.

Additionally, prompt design is rarely a one-shot activity; instead, it evolves through iterative refinement as developers observe model outputs and adjust their inputs accordingly. Developers must consider factors such as phrasing, structure, and level of detail, as even small changes can significantly affect outcomes. This results in a workflow centered around **progressive refinement and hypothesis-driven adjustments**, where prompts are continuously shaped to better align with desired outputs.

3.2.2 Implementation

Implementation in prompt programming differs significantly from traditional coding, as it involves integrating prompts into software systems rather than writing executable logic. Developers embed prompts within applications, often combining them with user inputs and system-level parameters to dynamically generate outputs. This requires developers to **treat prompts as functional components within the system**, managing how they are constructed, parameterized, and invoked during execution.

A key aspect of implementation is handling variability in model outputs. Since foundation models produce probabilistic responses, developers must design systems that can accommodate inconsistent or unexpected results. This may involve adding constraints, post-processing outputs, or structuring prompts in ways that reduce ambiguity. As a result, implementation focuses on **building robustness around inherently uncertain model behavior**, rather than ensuring strict correctness through deterministic logic.

Furthermore, implementation often includes mechanisms for chaining prompts or combining them with other computational components, enabling more complex workflows. Developers may design pipelines where outputs from one prompt feed into another, creating multi-step interactions with the model. This highlights how prompt programming extends beyond single interactions, requiring developers to **orchestrate sequences of prompt-based operations within larger system architectures**.

3.2.3 Debugging

Debugging in prompt programming is fundamentally different from traditional debugging due to the lack of explicit execution traces or deterministic behavior. Developers cannot step through code or inspect

intermediate states in a conventional sense; instead, they must rely on observing outputs and inferring the causes of errors. This makes debugging an exploratory process where developers **diagnose issues by iteratively modifying prompts and analyzing changes in outputs**, rather than identifying precise faults in code.

3.2.4 Data Curation

Data curation plays a crucial role in prompt programming, as the quality and structure of inputs provided to the model directly influence the outputs. Developers often curate examples, context, or reference data that are included within prompts to guide the model’s responses. This involves selecting relevant and representative information, ensuring that prompts are both informative and concise. As a result, developers must **carefully manage and structure input data to shape model behavior effectively**, making data curation an integral part of the development process.

3.2.5 Evaluation

Evaluation in prompt programming is inherently empirical, as developers assess the quality of outputs based on observed results rather than predefined correctness criteria. This involves testing prompts across multiple inputs and scenarios to determine whether they produce consistent and useful outputs. Developers must therefore rely on **iterative testing and qualitative judgment** to evaluate performance, often considering factors such as relevance, accuracy, and coherence.

Moreover, evaluation is closely tied to the iterative nature of prompt programming, as insights gained during evaluation inform subsequent refinements. Developers continuously cycle between testing and modifying prompts, using feedback to improve outcomes over time. This creates a loop where evaluation is not a final step but an ongoing process, emphasizing **continuous improvement through repeated experimentation and validation**.

4 Implications

One major implication is the need for new tools and abstractions tailored to prompt programming. Traditional development environments are designed for deterministic code and do not adequately support the iterative and experimental nature of prompt design. This suggests the need for systems that enable easier testing, comparison, and refinement of prompts, allowing developers to better understand and control model behavior. Consequently, there is a clear opportunity to **design tooling that supports iteration, visualization of outputs, and structured prompt management**.

Another implication lies in the skills required for developers. Prompt programming demands a different set of competencies compared to traditional coding, including the ability to craft effective natural language instructions and interpret model outputs. Developers must also develop intuition about how models behave, which is not typically part of standard programming education. This highlights the importance of **redefining developer skill sets to include prompt design, experimentation, and interaction with AI systems**, suggesting changes in both training and practice.

Finally, the discussion points to broader impacts on software engineering methodologies. Established practices such as testing, debugging, and verification must be rethought in the context of probabilistic systems. Since outputs cannot always be guaranteed, developers must adopt more flexible and empirical approaches to ensure system reliability. This leads to the need for **new methodologies that accommodate uncertainty, variability, and continuous evaluation**, reshaping how software systems are designed and maintained.

4.1 Prompt Programming

The discussion frames prompt programming as a fundamentally new programming paradigm that differs from traditional software development in both process and abstraction. Instead of writing deterministic code, developers interact with foundation models through prompts, leading to a workflow that is inherently exploratory and iterative. This paradigm shift emphasizes that programming is no longer

solely about specifying exact instructions but about **guiding model behavior through structured natural language**, where outcomes depend on both prompt design and model interpretation.

A key insight is that prompt programming introduces a tight feedback loop between the developer and the model, where outputs continuously inform subsequent inputs. This creates a dynamic interaction model in which developers must experiment, evaluate, and refine their prompts to achieve desired results. Unlike traditional debugging and optimization, this process is less about fixing errors and more about aligning outputs with expectations. As a result, prompt programming is characterized by **continuous iteration, empirical validation, and adaptive refinement**, making it closer to exploratory problem-solving than conventional programming.

The discussion also highlights that prompt programming shifts the locus of control from explicit logic to implicit behavior learned by models. Developers must work within the constraints of the model’s training and capabilities, leading to a form of programming that is both powerful and limited by its opacity. This requires a new mindset where developers **balance control and flexibility, leveraging model strengths while mitigating unpredictability**, ultimately redefining how software functionality is specified and implemented.

4.2 Takeaways

Overall, the paper emphasizes that prompt programming represents a significant evolution in software development, requiring new ways of thinking, working, and building systems. It underscores that developers must move beyond traditional paradigms and embrace a model of programming that is interactive, iterative, and probabilistic. The key takeaway is that **prompt programming is not just a tool-based enhancement but a paradigm shift that redefines the role of the programmer, the nature of code, and the process of software development**.

5 Threats to Validity

Internal Validity The authors acknowledge potential threats to internal validity arising from the qualitative nature of the study and the reliance on self-reported data from participants. Since the findings are derived from interviews, they may be influenced by participants’ ability to accurately recall and articulate their experiences. Additionally, researcher bias during coding and interpretation could affect how themes are identified and constructed. To mitigate these concerns, the study employs systematic grounded theory procedures, including iterative coding and constant comparison, ensuring that insights are **consistently derived from the data rather than imposed by the researchers**, thereby strengthening the credibility of the findings.

External Validity External validity is limited by the sample size and the specific characteristics of the participants involved in the study. While the authors include participants with diverse backgrounds and experience levels, the findings may not generalize to all developers or contexts, particularly those who have limited exposure to foundation models. Furthermore, as prompt programming is an emerging practice, the observed behaviors may evolve over time as tools and models improve. The authors therefore note that the results should be interpreted as capturing **current practices and trends rather than universally generalizable behaviors**, highlighting the need for further studies across broader populations and settings.

Conclusion Validity Threats to conclusion validity stem from the interpretive nature of qualitative analysis, where conclusions are drawn from patterns identified in the data rather than statistical inference. The absence of quantitative measures means that findings rely on the researchers’ ability to accurately synthesize themes and relationships. However, the use of grounded theory techniques, including systematic coding and validation through repeated analysis, helps ensure that conclusions are well-supported. This approach strengthens the reliability of the results by ensuring that **conclusions are grounded in recurring patterns and consistent observations across participants**, rather than isolated instances.

References

- [1] *ChatGPT*. <https://chatgpt.com/>. 2024.
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, et al. “GPT-4 Technical Report”. In: *arXiv preprint arXiv:2303.08774* (2023).
- [3] James Betker, Gabriel Goh, Li Jing, et al. “Improving Image Generation with Better Captions”. In: *OpenAI Report* (2023).
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, et al. “Evaluating Large Language Models Trained on Code”. In: *arXiv preprint arXiv:2107.03374* (2021).
- [5] Jenny T Liang, Chenyang Yang, and Brad A Myers. “A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges”. In: *IEEE/ACM International Conference on Software Engineering (ICSE)*. 2024, pp. 1–13. DOI: [10.1145/3597503.3608128](https://doi.org/10.1145/3597503.3608128).
- [6] Long Ouyang, Jeffrey Wu, Xu Jiang, et al. “Training Language Models to Follow Instructions with Human Feedback”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 27730–27744.
- [7] A. Strauss and J.M. Corbin. *Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory*. SAGE Publications, 1998. ISBN: 9780803959408. URL: <https://books.google.com/books?id=wTwYUnHYsmMC>.